Technical Specification: Adversarial Failure Discovery for Multi-Robot Fleets

1. Project Objective

Build a prototype system that automatically generates and evaluates adversarial scenarios against a publicly accessible multi-robot fleet controller.

The system should search for fleet-level failures such as:

- deadlocks;
- congestion collapse;
- task starvation;
- excessive task latency;
- traffic negotiation failures;
- shared-resource contention;
- recovery loops;
- cascading degradation after a robot failure;
- dependency on a single robot or resource.

The prototype must compare directed evolutionary search against random scenario sampling under an equivalent simulation budget.

The primary output is not a trained controller. The output is a set of:

- reproducible failure scenarios;
- deterministic replay packages;
- generalized failure categories;
- benchmark results comparing search methods;
- machine-readable metrics.

2. Reference Technology Stack

Use the following public stack:

- Ubuntu 24.04
- ROS 2 Kilted
- Open-RMF
- `rmf_demos`
- Nav2
- Gazebo Ionic
- Python 3.12+
- ROS 2 bag recording
- YAML and JSONL for scenario and event storage

Preferred repository:

```
https://github.com/open-rmf/rmf_demos
```

Use an existing Open-RMF demo world. Prefer a warehouse, manufacturing, office, or logistics environment with multiple robots and task dispatch support.

Do not write a new fleet controller for the first prototype.

Treat Open-RMF, Nav2, and Gazebo as the system under test.

3. System Architecture

The system should use the following top-level architecture:

```
Adversarial Search Engine
        ↓
Scenario Generator
        ↓
Simulation Adapter
        ↓
Open-RMF + Nav2 + Gazebo
        ↓
Telemetry and Event Collector
        ↓
Metrics and Failure Detector
        ↓
Fitness Score
        ↓
Search Engine Update
```

The adversarial testing system must remain external to Open-RMF and Nav2 wherever possible.

Avoid modifying upstream controller source code during the initial prototype.

4. Primary Components

Implement the following components.

4.1 Experiment Orchestrator

Responsible for:

- launching the simulation stack;
- checking service readiness;
- applying scenario parameters;
- submitting tasks;
- monitoring simulation progress;
- detecting completion or timeout;
- collecting metrics;
- exporting replay artifacts;
- shutting down or resetting the simulation;
- returning results to the optimizer.

Suggested module:

```
src/orchestrator/
```

Suggested interface:

```python
from typing import Any, Protocol


class SimulationAdapter(Protocol):
    def initialize(self) -> None:
        """Start or connect to the simulation environment."""

    def health_check(self) -> dict[str, Any]:
        """Return readiness status for required services and nodes."""

    def reset(self, seed: int, scenario: dict[str, Any]) -> None:
        """Reset the simulation and apply scenario parameters."""

    def start(self) -> None:
        """Begin or resume simulation execution."""

    def submit_tasks(self, tasks: list[dict[str, Any]]) -> None:
        """Submit fleet tasks to Open-RMF."""

    def observe(self) -> dict[str, Any]:
        """Return the latest system state and telemetry."""

    def is_complete(self) -> bool:
        """Return whether the mission has completed."""
```

```
    def has_failed(self) -> bool:
        """Return whether a qualifying failure has occurred."""

    def calculate_metrics(self) -> dict[str, float]:
        """Calculate final fleet and mission metrics."""

    def export_replay(self, output_dir: str) -> None:
        """Export all artifacts required to reproduce the run."""

    def shutdown(self) -> None:
        """Cleanly stop simulation processes."""
```

4.2 Scenario Generator

Responsible for converting a scenario genome into:

- robot configuration;
- initial robot positions;
- task workloads;
- task arrival timing;
- robot performance modifiers;
- infrastructure delays;
- resource availability;
- injected failures;
- communication or state-update degradation;
- simulation seed.

Suggested module:

```
src/scenarios/
```

The generator must validate all scenarios before execution.

Invalid scenarios should be rejected or repaired before reaching the simulator.

4.3 Search Engine

Support at least two search strategies:

1. Random search
2. Genetic algorithm

Design the interface so additional optimizers can be added later, including:

- CMA-ES;
- Bayesian optimization;

- novelty search;
- MAP-Elites;
- adversarial reinforcement learning.

Suggested interface:

```python
from typing import Any, Protocol


class SearchStrategy(Protocol):
    def suggest(self, count: int) -> list[dict[str, Any]]:
        """Return candidate scenario genomes."""

    def observe(
        self,
        scenarios: list[dict[str, Any]],
        results: list[dict[str, Any]],
    ) -> None:
        """Update the search strategy using evaluated results."""

    def state_dict(self) -> dict[str, Any]:
        """Return serializable optimizer state."""

    def load_state_dict(self, state: dict[str, Any]) -> None:
        """Restore optimizer state."""
```

Suggested modules:

```
src/search/random_search.py
src/search/genetic_search.py
src/search/base.py
```

4.4 Telemetry Collector

Collect the minimum information required to evaluate fleet behavior.

Capture:

- robot pose and velocity;
- robot task state;
- fleet task assignments;
- task submission and completion timestamps;
- battery state where available;
- traffic schedule information;
- negotiation events;

- Nav2 recovery events;
- planner and controller failures;
- robot idle time;
- robot blocked time;
- simulation clock;
- infrastructure state;
- injected failure events.

Suggested module:

```
src/telemetry/
```

Store events in JSONL:

```
{"timestamp": 14.2, "event": "task_submitted", "task_id": "task_17"}
{"timestamp": 19.8, "event": "task_assigned", "task_id": "task_17", "robot_id":
"robot_3"}
{"timestamp": 64.1, "event": "robot_blocked", "robot_id": "robot_3"}
{"timestamp": 120.0, "event": "task_timeout", "task_id": "task_17"}
```

4.5 Metrics Engine

Calculate fleet-level metrics for each run.

Required metrics:

```
task_completion_ratio
incomplete_task_ratio
mean_task_latency
p95_task_latency
maximum_task_latency
deadlock_duration
number_of_deadlock_events
task_starvation_count
negotiation_failure_count
recovery_event_count
recovery_loop_count
robot_failure_count
fleet_fragmentation_score
robot_idle_time_mean
robot_idle_time_variance
blocked_time_total
```

```
simulation_runtime
wall_clock_runtime
```

Suggested module:

```
src/metrics/
```

4.6 Failure Detector

Identify qualifying failure cases.

Initial failure classes:

```
DEADLOCK
CONGESTION_COLLAPSE
TASK_STARVATION
TASK_TIMEOUT
TRAFFIC_NEGOTIATION_FAILURE
RECOVERY_LOOP
RESOURCE_CONTENTION
CASCADING_ROBOT_FAILURE
FLEET_FRAGMENTATION
THROUGHPUT_COLLAPSE
SIMULATION_ERROR
UNKNOWN_FAILURE
```

A failure detector should emit:

```
{
    "is_failure": True,
    "failure_type": "DEADLOCK",
    "severity": 0.87,
    "confidence": 0.94,
    "evidence": {
        "blocked_robots": 5,
        "blocked_duration_seconds": 91.2,
        "unfinished_tasks": 7
    }
}
```

Suggested module:

```
src/failures/
```

4.7 Failure Clustering

Group similar failure scenarios into generalized categories.

The first implementation may use a simple feature vector and distance-based clustering.

Candidate features:

```
failure_type
incomplete_task_ratio
deadlock_duration
task_starvation_count
negotiation_failure_count
recovery_loop_count
robot_failure_count
blocked_time_total
task_arrival_rate
robot_count
speed_multiplier
door_delay
state_update_latency
failure_time
```

Possible first implementation:

- normalize numeric features;
- one-hot encode categorical failure types;
- use DBSCAN or hierarchical clustering;
- choose a representative scenario for each cluster;
- export cluster summaries.

Suggested module:

```
src/clustering/
```

Do not use an LLM for primary failure clustering in the first version.

A language model may later generate human-readable summaries after deterministic clustering is complete.

4.8 Replay Exporter

Every retained failure must be reproducible.

Each replay package should contain:

```
failure_case_00017/
├── scenario.yaml
├── seed.json
├── tasks.json
├── metrics.json
├── failure.json
├── events.jsonl
├── environment.json
├── optimizer.json
├── rosbag/
├── stdout.log
├── stderr.log
└── reproduce.sh
```

The `reproduce.sh` script should:

1. validate dependencies;
2. launch the same simulation world;
3. restore the same seed;
4. apply the same scenario;
5. submit the same tasks;
6. record output;
7. report whether the failure reproduced.

Suggested module:

```
src/replay/
```

5. Scenario Genome

Start with six to ten evolvable parameters.

Recommended initial genome:

```
seed: 1042

fleet:
```

```
  robot_count: 8
  max_speed_multiplier: 0.85
  acceleration_multiplier: 1.0

tasks:
  task_count: 20
  arrival_interval_seconds: 8.0
  priority_skew: 0.6
  pickup_region_bias: 0.5
  dropoff_region_bias: 0.5

facility:
  blocked_lane_id: null
  door_delay_seconds: 3.0
  charger_count: 2

faults:
  failed_robot_id: null
  failure_time_seconds: null
  state_update_latency_ms: 0
```

Initial evolvable parameters:

```
robot_count
max_speed_multiplier
task_count
arrival_interval_seconds
priority_skew
blocked_lane_id
door_delay_seconds
failed_robot_id
failure_time_seconds
state_update_latency_ms
```

Suggested ranges:

```
parameter_ranges:
  robot_count:
    type: integer
    min: 3
    max: 12

  max_speed_multiplier:
    type: float
    min: 0.4
```

```
  max: 1.2

task_count:
  type: integer
  min: 5
  max: 50

arrival_interval_seconds:
  type: float
  min: 1.0
  max: 30.0

priority_skew:
  type: float
  min: 0.0
  max: 1.0

blocked_lane_id:
  type: categorical
  values:
    - null
    - lane_01
    - lane_02
    - lane_03

door_delay_seconds:
  type: float
  min: 0.0
  max: 30.0

failed_robot_id:
  type: categorical
  values:
    - null
    - robot_1
    - robot_2
    - robot_3
    - robot_4

failure_time_seconds:
  type: float
  min: 10.0
  max: 300.0

state_update_latency_ms:
  type: integer
  min: 0
  max: 2000
```

Add conditional validation:

- `failure_time_seconds` is ignored when `failed_robot_id` is null.
- `blocked_lane_id` must reference an existing navigable lane.
- `robot_count` must not exceed the supported fleet configuration.
- all generated task locations must be reachable.
- scenario setup must not fail before mission execution begins.

6. Fitness Function

The optimizer should maximize failure severity.

Initial weighted score:

```
failure_score =
    3.0 × incomplete_task_ratio
  + 2.0 × normalized_p95_task_latency
  + 3.0 × normalized_deadlock_duration
  + 2.0 × normalized_task_starvation_count
  + 1.5 × normalized_negotiation_failure_count
  + 1.5 × normalized_recovery_loop_count
  + 1.0 × normalized_blocked_time_total
  + 1.0 × fleet_fragmentation_score
```

Clamp the final score to a known range such as:

```
0.0 to 10.0
```

Penalize invalid or unusable scenarios:

```
invalid_scenario_penalty = -10.0
simulation_startup_failure_penalty = -8.0
non_reproducible_failure_penalty = -5.0
```

Do not reward:

- simulator crashes caused by invalid configuration;
- impossible initial robot overlap;
- malformed tasks;
- missing dependencies;
- controller launch failures unrelated to scenario conditions.

Store both the composite score and all component metrics.

7. Deadlock Detection

Implement a first-pass deadlock detector.

A candidate deadlock exists when:

- two or more robots have active navigation tasks;
- their displacement remains below a configurable threshold;
- their commanded or intended motion remains active;
- the condition persists longer than a timeout;
- the mission is not complete.

Suggested defaults:

```
deadlock_detection:
  minimum_robots: 2
  movement_threshold_meters: 0.10
  observation_window_seconds: 15
  deadlock_timeout_seconds: 30
```

Distinguish deadlock from intentional waiting where possible.

Record:

- affected robots;
- start time;
- duration;
- current tasks;
- nearby robots;
- traffic negotiation state;
- relevant map region.

8. Task Starvation Detection

A task is starved when:

- it remains queued or assigned beyond a defined threshold;
- other newer tasks continue to complete;
- no valid resource constraint explains the delay;
- the mission has not terminated.

Suggested default:

```
task_starvation:
  threshold_seconds: 120
  minimum_newer_tasks_completed: 2
```

9. Recovery Loop Detection

Detect repeated Nav2 recovery behavior.

A recovery loop may exist when:

- the same robot enters recovery behavior repeatedly;
- the same navigation goal remains active;
- progress remains below the movement threshold;
- the recovery count exceeds a limit within a time window.

Suggested defaults:

```
recovery_loop:
  maximum_recoveries: 4
  window_seconds: 60
  progress_threshold_meters: 0.25
```

10. Simulation Lifecycle

Implement a deterministic simulation lifecycle.

Required flow:

```
1. Generate scenario
2. Validate scenario
3. Allocate run directory
4. Start ROS 2 and Gazebo processes
5. Wait for required nodes and services
6. Apply robot and environment configuration
7. Start telemetry collection
8. Submit task workload
9. Run until completion, failure, or timeout
10. Calculate metrics
11. Classify failure
12. Export replay
13. Stop ROS bag recording
14. Shut down simulation
```

```
15. Verify no orphan processes remain
16. Return result to optimizer
```

Each simulation run must use a unique run identifier.

Example:

```
run_2026-07-20T14-32-11_seed_1042_candidate_0031
```

11. Process Isolation

ROS 2 and Gazebo processes are prone to state leakage and orphan processes.

For the prototype, prefer one of these approaches:

Preferred

Run each candidate evaluation inside a disposable container.

Acceptable initial approach

Use a dedicated process group and fully terminate all spawned processes after every run.

The orchestrator must:

- capture process IDs;
- capture stdout and stderr;
- enforce startup timeouts;
- enforce execution timeouts;
- send graceful shutdown;
- force-kill remaining processes;
- verify no prior simulation nodes remain active.

Do not reuse a potentially corrupted simulator state unless reset reliability has been proven.

12. Benchmark Design

Compare random search and genetic search under identical budgets.

Required benchmark dimensions:

```
simulation count
wall-clock time
number of random seeds
```

```
controller version
world version
scenario parameter ranges
maximum mission runtime
compute configuration
```

Minimum benchmark:

```
benchmark:
  methods:
    - random_search
    - genetic_search

  independent_runs_per_method: 10
  evaluations_per_run: 500
  mission_timeout_seconds: 300
  fixed_world: true
  equal_compute_budget: true
```

Required benchmark metrics:

```
time_to_first_failure
evaluations_to_first_failure
best_failure_score
unique_failure_count
unique_failure_categories
failure_reproduction_rate
invalid_scenario_rate
mean_failure_severity
failure_diversity
wall_clock_cost_per_failure
```

Primary benchmark metric:

```
Unique reproducible failure categories discovered per 1,000 simulations
```

Secondary metric:

```
Evaluations required to discover the first severe failure
```

13. Genetic Algorithm Specification

Implement a simple genetic algorithm first.

Suggested defaults:

```
genetic_algorithm:
  population_size: 40
  generations: 25
  elite_fraction: 0.10
  tournament_size: 3
  crossover_probability: 0.70
  mutation_probability: 0.25
  random_injection_fraction: 0.10
```

Mutation behavior:

- numeric parameters: Gaussian or bounded perturbation;
- integer parameters: bounded integer mutation;
- categorical parameters: random alternative selection;
- conditional parameters: mutate parent and dependent fields together.

Crossover behavior:

- uniform crossover is acceptable;
- preserve valid parameter combinations;
- repair invalid children before evaluation.

Include diversity preservation:

- inject random candidates every generation;
- penalize duplicate genomes;
- track hash of normalized scenario configurations.

14. Random Search Specification

Random search must sample from the same parameter ranges and validation rules as the genetic algorithm.

Use reproducible seeds.

Do not give the genetic algorithm access to scenario parameters unavailable to random search.

15. Failure Uniqueness

Do not count minor variants of the same failure as separate discoveries.

Create a failure signature:

```
{
    "failure_type": "DEADLOCK",
    "affected_robot_count": 4,
    "map_region": "north_corridor",
    "task_state_pattern": "delivery_delivery_patrol",
    "resource_state": "door_2_delayed",
    "causal_features": [
        "high_task_arrival_rate",
        "slow_robot",
        "blocked_lane"
    ]
}
```

For the first implementation, define uniqueness using:

- failure type;
- affected map region;
- affected robot count bucket;
- resource involved;
- task state pattern;
- metric vector distance.

Store exact scenarios separately even when they belong to the same generalized category.

16. Repository Structure

Use the following structure:

```
adversarial-fleet-testing/
├── README.md
├── pyproject.toml
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
├── configs/
│   ├── default.yaml
│   ├── search_random.yaml
│   ├── search_genetic.yaml
```

```
│   ├── metrics.yaml
│   └── benchmark.yaml
├── src/
│   └── adversarial_fleet/
│       ├── __init__.py
│       ├── cli.py
│       ├── orchestrator/
│       │   ├── base.py
│       │   ├── process_manager.py
│       │   └── rmf_adapter.py
│       ├── scenarios/
│       │   ├── genome.py
│       │   ├── generator.py
│       │   ├── validation.py
│       │   └── task_generator.py
│       ├── search/
│       │   ├── base.py
│       │   ├── random_search.py
│       │   └── genetic_search.py
│       ├── telemetry/
│       │   ├── collector.py
│       │   ├── ros_topics.py
│       │   └── event_writer.py
│       ├── metrics/
│       │   ├── calculator.py
│       │   └── normalization.py
│       ├── failures/
│       │   ├── detector.py
│       │   ├── deadlock.py
│       │   ├── starvation.py
│       │   └── recovery_loop.py
│       ├── clustering/
│       │   ├── features.py
│       │   └── cluster.py
│       ├── replay/
│       │   ├── exporter.py
│       │   └── reproduce.py
│       └── benchmark/
│           ├── runner.py
│           ├── analysis.py
│           └── report.py
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├── scripts/
```

```
│   ├── install_dependencies.sh
│   ├── launch_demo.sh
│   ├── run_candidate.sh
│   ├── run_benchmark.sh
│   └── reproduce_failure.sh
├── results/
└── docs/
    ├── architecture.md
    ├── scenario_schema.md
    └── benchmark_methodology.md
```

17. Command-Line Interface

Provide a CLI.

Required commands:

```
aft health-check
aft run-scenario --scenario configs/example_scenario.yaml
aft search --config configs/search_genetic.yaml
aft benchmark --config configs/benchmark.yaml
aft replay results/failures/failure_case_00017
aft cluster results/runs/
aft report results/benchmark_001/
```

Expected behavior:

Health check

```
aft health-check
```

Checks:

- ROS 2 installation;
- required packages;
- Gazebo availability;
- RMF demo package;
- required ROS topics and services;
- writable output directory;
- Docker availability where configured.

Run scenario

```
aft run-scenario --scenario scenario.yaml
```

Runs one deterministic candidate and exports results.

Search

```
aft search --config configs/search_genetic.yaml
```

Runs the configured search strategy.

Benchmark

```
aft benchmark --config configs/benchmark.yaml
```

Runs all methods using equivalent budgets.

Replay

```
aft replay results/failures/failure_case_00017
```

Attempts deterministic reproduction.

18. Configuration

Use YAML configuration with schema validation.

Example top-level config:

```
project:
  name: adversarial-fleet-testing
  output_dir: ./results

simulation:
  adapter: rmf_demo
  world: selected_demo_world
  mission_timeout_seconds: 300
  startup_timeout_seconds: 120
  deterministic: true

recording:
```

```
  enable_rosbag: true
  enable_video: false
  record_stdout: true
  record_stderr: true

search:
  strategy: genetic
  evaluations: 1000
  parallel_workers: 1

failure_detection:
  deadlock: true
  starvation: true
  recovery_loop: true
  throughput_collapse: true
```

Use Pydantic or an equivalent typed validation library.

19. Testing Requirements

Unit tests

Test:

- scenario validation;
- genome mutation;
- crossover;
- deterministic random generation;
- fitness calculation;
- metric normalization;
- deadlock detection;
- failure signature generation;
- replay package creation.

Integration tests

Test:

- simulation launch;
- readiness detection;
- task submission;
- metric collection;
- timeout handling;
- process cleanup;
- deterministic scenario reproduction.

Failure-injection test

Create at least one intentionally pathological scenario that reliably produces a known failure.

Use it to validate:

- detector behavior;
- replay export;
- reproduction;
- metric calculation.

20. Logging

Use structured logging.

Every log entry should include:

```
run_id
candidate_id
generation
search_method
simulation_time
wall_clock_time
event_type
```

Example:

```
{
  "run_id": "benchmark_001",
  "candidate_id": "candidate_031",
  "generation": 4,
  "search_method": "genetic",
  "event_type": "failure_detected",
  "failure_type": "DEADLOCK",
  "severity": 0.87
}
```

21. Performance

Do not optimize prematurely.

Initial target:

```
One simulation worker
One candidate at a time
Reliable reset and replay
```

After correctness is established, allow parallel workers.

Parallelization must isolate:

- ROS domain IDs;
- network ports;
- simulation processes;
- temporary directories;
- output files.

22. Security and Deployment Constraints

The long-term product should support on-premises and air-gapped deployment.

The prototype should therefore avoid unnecessary cloud dependencies.

Requirements:

- no mandatory telemetry;
- no external API dependency;
- no SaaS requirement;
- local search and analysis;
- local artifact storage;
- containerized deployment;
- configuration through local files;
- deterministic offline execution.

Do not upload proprietary controllers, customer simulations, failure cases, vulnerabilities, or sensitive operational information to public model APIs.

For the public prototype, use only public repositories, synthetic scenarios, and non-sensitive configurations.

23. First 48-Hour Implementation Scope

Prioritize the following.

Phase A: Environment

1. Install or containerize ROS 2, Gazebo, Open-RMF, Nav2, and `rmf_demos`.
2. Launch one demo world manually.
3. Confirm robots can accept tasks.

4. Identify relevant ROS topics, actions, and services.
5. Confirm the simulation can be stopped and relaunched reliably.

Phase B: Single Scenario

1. Define one scenario YAML file.
2. Programmatically launch the world.
3. Submit a fixed task workload.
4. collect task state and robot state;
5. stop after mission completion or timeout;
6. export metrics and logs.

Phase C: Parameter Variation

Support three to five parameters initially:

```
robot_count
task_count
arrival_interval_seconds
max_speed_multiplier
door_delay_seconds
```

Do not begin with every proposed failure parameter.

Phase D: Random Search

1. Generate random valid scenarios.
2. Evaluate each scenario.
3. Track best failure score.
4. Save all qualifying failures.

Phase E: Genetic Search

1. Create initial population.
2. Evaluate population.
3. rank candidates;
4. select parents;
5. apply crossover and mutation;
6. run several generations;
7. save optimizer state.

Phase F: Benchmark

Run random and genetic search under the same number of evaluations.

Produce:

```
best score by evaluation
unique failures by evaluation
time to first failure
failure categories found
invalid scenario count
```

Phase G: Replay

For the top five failures:

1. export scenario and seed;
2. export logs and metrics;
3. generate `reproduce.sh`;
4. rerun each case;
5. record whether it reproduced.

24. Initial Success Criteria

The prototype is successful when:

- one Open-RMF demo can be launched programmatically;
- tasks can be submitted automatically;
- at least three scenario parameters can be varied;
- metrics are collected without manual intervention;
- random search completes a full benchmark run;
- genetic search completes a full benchmark run;
- at least one fleet-level failure is detected;
- retained failures can be replayed;
- the benchmark produces machine-readable and visual results.

The technical thesis receives preliminary support when:

- genetic search finds severe failures using fewer evaluations than random search;
- genetic search discovers more unique failure categories under the same budget;
- failures reproduce consistently;
- failures are caused by fleet behavior rather than invalid simulator configuration.

25. Kill Conditions

Stop or redesign the approach if:

- simulation reset is too unreliable for automated campaigns;
- individual runs require excessive manual intervention;
- generated scenarios mostly crash the simulator rather than stress the controller;
- the fitness function rewards trivial invalid configurations;
- evolutionary search repeatedly finds only one duplicated failure;

- random search performs similarly under equivalent budgets;
- failures cannot be deterministically replayed;
- each scenario requires extensive controller modification.

26. Deferred Features

Do not implement during the initial prototype:

- graphical dashboard;
- authentication;
- cloud orchestration;
- billing;
- multi-tenant support;
- hardware-in-the-loop;
- formal verification;
- LLM-generated scenarios;
- video rendering for every run;
- high-fidelity factory modeling;
- universal simulator support;
- customer-facing report design;
- government compliance certification;
- distributed GPU infrastructure.

27. Codex Execution Instructions

Start by inspecting the selected `rmf_demos` environment and documenting:

1. exact launch command;
2. required ROS 2 packages;
3. available robots and fleets;
4. task submission interface;
5. relevant state topics;
6. relevant traffic and negotiation topics;
7. reset and shutdown behavior;
8. whether deterministic seeding is supported;
9. whether robot speed and infrastructure delays can be configured externally.

Then create the repository structure and implement the smallest vertical slice:

```
scenario.yaml
    ↓
launch simulation
    ↓
submit tasks
    ↓
collect metrics
```

```
    ↓
calculate score
    ↓
export replay package
```

Do not implement the genetic algorithm until one deterministic scenario can run end to end.

After the vertical slice works:

1. implement random scenario generation;
2. run ten deterministic test scenarios;
3. verify cleanup and replay;
4. implement the genetic algorithm;
5. run equal-budget benchmarks;
6. export raw data and charts.

When upstream APIs are unclear, inspect installed ROS interfaces and the source code rather than inventing interfaces.

Use typed Python, explicit error handling, structured logs, unit tests, and configuration validation.

Favor correctness and reproducibility over abstraction or polish.